

testes.c

```
#include <stdio.h>
#include <stdlib.h>
#include <CUnit/CUnit.h>
#include "Jogo.h"
```

```
//
//
// Funções de Teste da Lógica
//
//
```

// Função que verifica se o baralho é criado corretamente com as 52 cartas

```
void test_criarBaralho(void) {
    jogo j;
    criarBaralho(&j);
    CU_ASSERT(j.baralho[0].num == 1);
    CU_ASSERT(j.baralho[0].naipe == 0);
    CU_ASSERT(j.baralho[51].num == 13);
    CU_ASSERT(j.baralho[51].naipe == 3);
}
```

1ª Carta é As

← Espadas

← Rei

← Espadas

↑
Não temos de testar todas as cartas. Se a primeira e a última estiverem corretas, as outras também estão

Testa se → // Função que testa se as cartas são baralhadas mantendo todas no baralho

manhuma carta desafiadas void test_baralharCartas(void) {

Alguns pontos jogo j;

criarBaralho(&j);

carta primeira_antes = j.baralho[0];

baralharCartas(&j);

int encontrou = 0; ← Começa falso

for (int i = 0; i < 52; i++) {

if (j.baralho[i].num == primeira_antes.num && j.baralho[i].naipe ==

primeira_antes.naipe)

encontrou = 1;

}

CU_ASSERT(encontrou == 1); ← O CU_ASSERT exige que o valor seja 1

}

↑
Testa todas as cartas para tentar encontrar o As de espadas

// Função que testa a distribuição inicial das cartas pelas 10 pilhas

```
void test_distribuirCartas(void) {
    jogo j;
    criarBaralho(&j);
    baralharCartas(&j);
    distribuirCartas(&j);
    CU_ASSERT(j.quantidade_pilhas[0] == 8);
    CU_ASSERT(j.quantidade_pilhas[3] == 7);
    CU_ASSERT(j.quantidade_pilhas[4] == 6);
    CU_ASSERT(j.quantidade_pilhas[9] == 1);
    CU_ASSERT(j.pilhas_concluidas == 0);
}
```

← Testar o início

← Primeira trilha

← Rei da mesa

← Última coluna

// Função que testa se o jogo é inicializado corretamente

```
void test_comecarJogo(void) {
    jogo j;
    começarJogo(&j);
    CU_ASSERT(j.pilhas_concluidas == 0);
    CU_ASSERT(j.quantidade_pilhas[0] == 8);
}
```

↑
Verifica valores iniciais

```

CU_ASSERT(j.quantidade_pilhas[9] == 1);
}

// Função que testa a validação dos limites de origem, destino e quantidade
void test_limitesValidos(void) {
    jogo j;
    comecarJogo(&j);
    CU_ASSERT(limitesValidos(&j, 0, 1, 1) == 1);
    CU_ASSERT(limitesValidos(&j, -1, 1, 1) == 0); ← Coluna inexistente (limite esquerdo)
    CU_ASSERT(limitesValidos(&j, 0, 10, 1) == 0); ← Coluna inexistente (limite direito)
    CU_ASSERT(limitesValidos(&j, 0, 1, 0) == 0); ← Não o Cartes
    CU_ASSERT(limitesValidos(&j, 0, 1, 100) == 0); ← Não muita cartas
}

// Função que testa se uma sequência de cartas é válida (mesmo naipe, ordem
// descendente)
void test_sequenciaValida(void) {
    jogo j;
    comecarJogo(&j);
    j.quantidade_pilhas[0] = 3;
    j.pilhas[0][0] = (carta){5, 0};
    j.pilhas[0][1] = (carta){4, 0};
    j.pilhas[0][2] = (carta){3, 0}; } → Enia uma sequência válida
    CU_ASSERT(sequenciaValida(&j, 0, 3) == 1); } Logo tem de retornar 1
    j.pilhas[0][1] = (carta){4, 1}; ← Altera o naipe
    CU_ASSERT(sequenciaValida(&j, 0, 3) == -2); ← tem de dar o resultado erro
    j.pilhas[0][1] = (carta){4, 0}; ← Coloca o naipe Correto
    j.pilhas[0][2] = (carta){2, 0}; ← desordena
    CU_ASSERT(sequenciaValida(&j, 0, 3) == -3); } tem de dar esse erro
}

// Função que testa se o destino aceita a carta a mover (valor
// imediatamente superior)
void test_destinoValido(void) {
    jogo j;
    comecarJogo(&j);
    j.quantidade_pilhas[1] = 1;
    j.pilhas[1][0] = (carta){6, 0}; } → Condição possível
    carta movida = {5, 0};
    CU_ASSERT(destinoValido(&j, movida, 1) == 1);
    carta invalida = {3, 0}; ← Carta que não é imediatamente a baixo
    CU_ASSERT(destinoValido(&j, invalida, 1) == -4); } tem de dar esse erro
    j.quantidade_pilhas[2] = 0;
    CU_ASSERT(destinoValido(&j, movida, 2) == 1); } → testa se a jogada move com
    sucesso para uma Coluna vazia
}

// Função que testa a validação completa de uma jogada combinando todos os
// critérios
void test_validarJogadaTotal(void) {
    jogo j;
    comecarJogo(&j);
    j.quantidade_pilhas[0] = 2;
    j.pilhas[0][0] = (carta){5, 0};
    j.pilhas[0][1] = (carta){4, 0};
    j.quantidade_pilhas[1] = 1;

```

```

j.pilhas[1][0] = (carta){6, 0};
CU_ASSERT(validarJogadaTotal(&j, 0, 1, 2) == 1); // testa a jogada anterior que é válida
CU_ASSERT(validarJogadaTotal(&j, -1, 1, 1) == -1); // não deu um limite inválido
j.pilhas[0][1] = (carta){4, 1}; // troca a carta
CU_ASSERT(validarJogadaTotal(&j, 0, 1, 2) == -2); // não posso mover por naipes ≠
}

```

// Função que testa se as cartas são movidas corretamente entre pilhas

```

void test_moverCartas(void) {
    jogo j;
    comecarJogo(&j);
    j.quantidade_pilhas[0] = 2;
    j.pilhas[0][0] = (carta){5, 0};
    j.pilhas[0][1] = (carta){4, 0};
    j.quantidade_pilhas[1] = 1;
    j.pilhas[1][0] = (carta){6, 0};
    moverCartas(&j, 0, 1, 2); // mao da cartas
    CU_ASSERT(j.quantidade_pilhas[0] == 0);
    CU_ASSERT(j.quantidade_pilhas[1] == 3);
    CU_ASSERT(j.pilhas[1][1].num == 5);
    CU_ASSERT(j.pilhas[1][2].num == 4);
}

```

→ Jogada válida

→ testa se as cartas foram corretamente movidas

// Função que testa a remoção automática de uma sequência completa (Rei ao

As) → também testa que só por estarem 13 cartas juntas a suíte não assume como pilha concluída

```

void test_tentarRemoverSequencia(void) {
    jogo j;
    comecarJogo(&j);
    j.pilhas_concluidas = 0;
    j.quantidade_pilhas[0] = 13;
    for (int i = 0; i < 13; i++) {
        j.pilhas[0][i].num = 13 - i;
        j.pilhas[0][i].naipe = 0;
    }
    tentarRemoverSequencia(&j, 0); // tenta remover i tem de ter sucesso
    CU_ASSERT(j.quantidade_pilhas[0] == 0);
    CU_ASSERT(j.pilhas_concluidas == 1);
    j.quantidade_pilhas[1] = 13;
    for (int i = 0; i < 13; i++) {
        j.pilhas[1][i].num = 13 - i;
        j.pilhas[1][i].naipe = 1;
    }
    j.pilhas[1][12].num = 5; // trocamos a carta do topo tornando impossível remover
    tentarRemoverSequencia(&j, 1); // logo tem de a remover
    CU_ASSERT(j.pilhas_concluidas == 1); // o n° de pilhas
}

```

Cria uma pilha aderada

Criamos outra sequência "perfeita"

Logo tem de a remover

o n° de pilhas

// Função que testa a execução completa de um movimento válido e inválido

```

void test_executarMovimento(void) {
    jogo j;
    comecarJogo(&j);
    j.quantidade_pilhas[0] = 1;
    j.pilhas[0][0] = (carta){4, 0};
    j.quantidade_pilhas[1] = 1;
    j.pilhas[1][0] = (carta){5, 0};
}

```

```
CU_ASSERT(executarMovimento(&j, 0, 1, 1) == 1); ← movimento válido
CU_ASSERT(j.quantidade_pilhas[0] == 0);
CU_ASSERT(j.quantidade_pilhas[1] == 2);
CU_ASSERT(executarMovimento(&j, -1, 1, 1) == -1); ← movimento inválido
```

```
}
```

```
//
```

```
//
```

```
// Funções Auxiliares para o CUnit
```

```
//
```

```
//
```

```
// Função que adiciona todos os testes à suite
```

```
void adicionarTestes(CU_pSuite suite) {
```

```
    CU_add_test(suite, "criarBaralho", test_criarBaralho);
    CU_add_test(suite, "baralharCartas", test_baralharCartas);
    CU_add_test(suite, "distribuirCartas", test_distribuirCartas);
    CU_add_test(suite, "comecarJogo", test_comecarJogo);
    CU_add_test(suite, "limitesValidos", test_limitesValidos);
    CU_add_test(suite, "sequenciaValida", test_sequenciaValida);
    CU_add_test(suite, "destinoValido", test_destinoValido);
    CU_add_test(suite, "validarJogadaTotal", test_validarJogadaTotal);
    CU_add_test(suite, "moverCartas", test_moverCartas);
    CU_add_test(suite, "tentarRemoverSequencia", test_tentarRemoverSequencia);
    CU_add_test(suite, "executarMovimento", test_executarMovimento);
```

```
}
```

Adiciona todos os testes à "suite"

Função → // Função que cria a suite de testes
maneira int configurarSuite(void) { // devolve o ponteiro

O erro pode acontecer por ter a memória cheia ou erro a iniciar o CUnit

```
    CU_pSuite suite = CU_add_suite("Suite_Logica", NULL, NULL);
    if (NULL == suite) { ← Se o suite não foi criado
        CU_cleanup_registry(); // limpa o registro do CUnit e dá erro
        return 0;
    }
    adicionarTestes(suite); // Se tiver 'OK'
    return 1; // adiciona os testes
```

```
}
```

→ Não tem setup nem teardown porque isolamos os testes e não usamos um ficheiro exterior com testes

Função → // Função que verifica na lista do CUnit se um teste específico falhou Quando uma

maneira int verificarSeFalhou(CU_pTest pTest) {
int falhou = 0; ← nº de testes falhados

Ponteiro para 1º erro

```
    CU_pFailureRecord pFailure = CU_get_failure_list();
    while (pFailure != NULL && falhou == 0) { ← Enquanto houver erros
        if ((*pFailure).pTest == pTest) // Se encontramos o que falhou
            falhou = 1; ← Se o endereço atual for igual ao do que falhou
        pFailure = (*pFailure).pNext; // Passa para o próximo teste
    }
    return falhou;
```

```
}
```

Retorna que falhou

Função → // Função que imprime o resultado de cada teste individualmente

maneira void imprimirTestesDaSuite(CU_pSuite pSuite) {
CU_pTest pTest = (*pSuite).pTest; ← Começa no 1º teste
while (pTest != NULL) {

← enquanto houver testes

```

if (verificarSeFalhou(pTest) == 1)
    printf("Funcao %s: NAO PASSOU\n", (*pTest).pName);
else
    printf("Funcao %s: PASSOU\n", (*pTest).pName);
pTest = (*pTest).pNext;
}
}

```

Se a função falhou, imprimir que falhou
Senão imprimir que passou

Função → *manida* // Função que percorre as suites e inicia a impressão dos resultados

```

void imprimirResultados(void) {
    printf("\n");
    CU_pSuite pSuite = (*CU_get_registry()).pSuite;
    while (pSuite != NULL) {
        imprimirTestesDaSuite(pSuite);
        pSuite = (*pSuite).pNext;
    }
    printf("\n");
}
//
//
// Função Main
//
//

```

Vai ao registro e pega na suite
Enquanto houver testes
Imprimir o Resultado
Passa para o próximo teste

- Fizemos desta maneira para que caso o enumerado mude e os testes também, seja mais fácil alterar
- Fizemos para respeitar a arquitetura do CUnit que não guarda os resultados em arrays logo não podemos fazer ++. O CUnit usa um ponteiro chamado pNext que aponta para o próximo elemento até ao nulo, 'NULL'

Função → *manida* int main(void) {

```

if (CUE_SUCCESS != CU_initialize_registry())
    return CU_get_error();

if (configurarSuite() == 0)
    return CU_get_error();

CU_run_all_tests();
imprimirResultados();

CU_cleanup_registry();
return CU_get_error();
}

```

Se não conseguir configurar a suite daí erro
Ao iniciar o registro, se não conseguir, daí erro;
Se não conseguir configurar a suite daí erro
Faz teste
Imprimir resultado
Limpa o registro para libertar a memória
Devolver o erro caso exista